PREDICTING AIRLINE DEPARTURE DELAYS WITH APACHE SPARK

Big Data Technologies,

Achintya, M.S. Data Science

Predicting Flight Delays

Agenda

Problem & Objective
Data & Big Data justification
ETL + leakage prevention
EDA insights
Feature engineering ("Gold pipeline")
Modeling + evaluation framework
Scaling analysis (in/out)
Conclusions + future scope

Massive Data Handling

Analyzing 6.5 million flight records requires robust distributed computing to ensure scalability and timely predictions.

Long-Tail Delay Patterns

Flight delays exhibit a long-tail distribution since most flights are on time, but a few experience significant delays.

Identifying High-Risk Flights

The focus is to spot flights most likely to be delayed for operational intervention, rather than just improving accuracy.

Problem Statement

- Flight delays lead to financial losses, operational disruptions, and poor passenger experience, so airlines require reliable methods to anticipate delays before departure rather than reacting after they occur.
- Most flights depart on time, making delayed flights a relatively rare outcome, which results in a highly imbalanced prediction problem that is difficult to model accurately.
- Delay behaviour varies significantly across seasons, airports, and airlines, introducing strong non-linear and temporal patterns in the data.
- The dataset is extremely large, containing millions of flight records across the United States, which makes traditional single-machine data processing approaches impractical.
- The prediction system must rely solely on pre-departure information to avoid data leakage, ensuring that the solution reflects realistic operational deployment conditions.

Objectives

- Develop a scalable machine learning framework to predict flight delays using large-scale aviation data, ensuring efficient processing of millions of flight records.
- Integrate flight operations, weather, and airport-level information into a unified modelling pipeline to capture both operational and environmental drivers of delays.
- Design data preprocessing and feature engineering steps that strictly prevent information leakage by using only pre-departure variables.
- Evaluate model performance using metrics suitable for highly imbalanced classification problems, prioritizing PR-AUC and Recall@Top-K over simple accuracy.
- Analyse the impact of scaling up the dataset and scaling out cluster resources to identify the optimal balance between predictive performance and computational efficiency.

Dataset Overview

- The final cleaned dataset contains over 7 million flight records, representing large-scale airline operations across the United States.
- Each flight is described using 35 engineered features, capturing temporal, operational, weather, and airport-related information.
- The dataset includes both categorical and numerical variables, requiring appropriate encoding and preprocessing before modeling.
- A stratified train–test split is applied to preserve the original delay class distribution across both datasets.
- This structure ensures the data is suitable for scalable machine learning while maintaining representativeness and statistical integrity.

```
Train Shape: (4968224, 35)
Test Shape: (2129240, 35)
```

	MONTH	DAY_OF_MONTH	DAY_OF_WEEK	TAIL_NUM	DEST_AIRPORT_ID	CRS_DEP_TIME	DEP_DEL15	DEP_TIME_BLK	CRS_ARR_TIME	CRS_ELAPSED_TIME	...	DEPARTING_AIRPORT	LATITUDE	LONGITUDE	PREVIOUS_AIRPORT	DATE	PRCP	SNOW	SNWD	TMAX	AWND
5931780	11	21	4	N717SA	15304	1950.0	0	1900-1959	2205	135.0	...	Indianapolis Muni/Weir Cook	39.729	-86.282	William P Hobby	2019-11-21	0.08	0.0	0.0	52.0	13.65
6728269	12	6	5	N418SW	10397	1234.0	1	1200-1259	1523	109.0	...	Shreveport Regional	32.447	-93.826	Atlanta Municipal	NaT	NaN	NaN	NaN	NaN	NaN
5491946	11	12	2	N110SY	12266	715.0	0	0700-0759	830	75.0	...	Dallas Fort Worth Regional	32.894	-97.030	NONE	2019-11-12	0.00	0.0	0.0	39.0	11.41
1183530	3	8	5	N955UW	12953	1200.0	0	1200-1259	1321	81.0	...	Logan International	42.364	-71.006	Philadelphia International	2019-03-08	0.00	0.0	0.0	38.0	10.96
6457496	12	5	4	N345NB	11775	1950.0	0	1900-1959	2132	162.0	...	Atlanta Municipal	33.641	-84.427	Roanoke Regional/Woodrum Field	2019-12-05	0.00	0.0	0.0	65.0	5.82

5 rows × 35 columns

Key Contributions

Build a distributed ML system that:

- Ingests and cleans multi-million-row flight records
- Prevents data leakage by excluding post-flight attribution variables
- Trains predictive models and evaluates with PR-focused metrics
- Validates behavior with ablation + clustering + anomaly detection
- Tests scaling behavior under varying data and cluster sizes

Key Dimensions:

•**Temporal:** FL_DATE, CRS_DEP_TIME (Scheduled), DEP_TIME (Actual).
•**Operational:** OP_CARRIER (Airline), TAIL_NUM (ID), ORIGIN, DEST.
•**Target:** DEP_DELAY (Continuous variable).

Feature Space & Target

•Started with 35 raw attributes, engineered to 26 features
•Key dimensions:
- Temporal: FL_DATE, scheduled vs actual departure
- Operational: carrier, tail number, origin/dest

•Target variable basis: departure delay minutes (DEP_DELAY)
•Modeling objective framed operationally as delay risk detection (imbalance handled later)

Critical Step: Leakage Prevention

•The Problem: Certain columns are populated after a flight completes, allowing the model to "peek" at the answer.
•Action: Systematically removed post-flight attribution columns:
- CANCELLATION_CODE, CARRIER_DELAY, WEATHER_DELAY, NAS_DELAY, SECURITY_DELAY, LATE_AIRCRAFT_DELAY.

•Result: Enforces a realistic prediction environment relying solely on schedule and network state.

Data Pre-Processing & ETL

- Flight operations data, weather data, and reference lookup tables are integrated into a unified dataset to provide comprehensive contextual information for each flight.
- Weather observations are temporally aligned with flights at the airport–day level, ensuring that environmental conditions correspond to the correct departure context.
- All preprocessing and transformations are performed using PySpark, enabling efficient handling of millions of records in a distributed environment.
- Data joins, cleaning, and feature generation are designed to minimize computational overhead while maintaining data consistency and accuracy.
- The ETL pipeline produces a clean, analysis-ready dataset that is optimized for scalable machine learning and model training.

ETL Overview

- Raw ingestion → schema/type enforcement
- Leakage prevention (drop post-flight explanatory fields)
- Null handling / cancellations removed
- Categorical indexing for high-cardinality columns
- Time parsing + feature extraction readiness (All optimized for distributed execution)

```
Processing /content/ONTIME_REPORTING_01.csv...
Sanitizing IDs, Dates, and Columns...
Dropping NaNs from Dep Time, Tail Num. Dropping Cancellations.
Adding Flight Number Sequence - SEGMENT_NUMBER...
Adding Concurrent Flights - CONCURRENT_FLIGHTS
Adding Scheduled Departure Hour - DEP_HOUR
Applying seat counts to flights - NUMBER_OF_SEATS
Applying Carrier Names - CARRIER_NAME
Adding flight statistics for carrier and airport...
Adding passenger statistics for carrier and airport...
Adding employee statistics for carrier...
Calculate Fleet Age - PLANE_AGE
Adding airport coordinates - LATITUDE, LONGITUDE, DEPARTING_AIRPORT
Adding airports - PREVIOUS_AIRPORT
Dropping bottom 10% of airports
Adding daily weather data...
Clean up unneeded columns
Cleaning up data types
Elapsed Time: 3.477111339569092
FINISHED
Processing /content/ONTIME_REPORTING_02.csv...
Sanitizing IDs, Dates, and Columns...
Dropping NaNs from Dep Time, Tail Num. Dropping Cancellations.
Adding Flight Number Sequence - SEGMENT_NUMBER...
Adding Concurrent Flights - CONCURRENT_FLIGHTS
...
```

Null Handling & Integrity Rules

•Dropped rows with nulls in DEP_TIME or ARR_TIME
•Interpretation: these typically reflect **cancelled flights**
•Rationale: project focuses on **predicting delay for operating flights** (consistent target)

High-Cardinality Handling (ETL Layer)

•Applied string indexing to:
•OP_CARRIER, ORIGIN, DEST
•Why:

- Hundreds of unique values (airports/carriers)
- Required for Spark ML feature assembly (VectorAssembler workflow)

Type Casting & Time Parsing

•Converted FL_DATE strings → timestamp (enables Day-of-Week / Month extraction)
•Cast DEP_DELAY and ARR_DELAY to integers for modeling
•Output: clean, typed dataset ready for distributed feature engineering

Data Leakage Prevention

- Post-flight delay attribution variables such as carrier, weather, and late aircraft delays are explicitly removed to prevent the model from accessing outcome-dependent information.
- Cancellation indicators and arrival-related timing fields are dropped to ensure that only information available before departure is used during prediction.
- Features related to passenger handling and revenue metrics are excluded, as they are not known at the time of departure and could introduce indirect leakage.
- Leakage prevention is applied after all data merges, ensuring that no derived features unintentionally encode post-event information.
- This strict leakage control guarantees realistic model evaluation and prevents artificially inflated performance metrics.

```
monthly_data.drop(columns = [
                'CANCELLED', 'CANCELLATION_CODE',
                'CARRIER_DELAY', 'WEATHER_DELAY', 'NAS_DELAY', 'SECURITY_DELAY', 'LATE_AIRCRAFT_DELAY',
               'Unnamed: 32',  'ARR_TIME_BLK', 'DEST_CITY_NAME',  'OP_CARRIER_FL_NUM',  'OP_UNIQUE_CARRIER',
                   'AIRLINE_ID',
                  'ORIGIN_CITY_NAME',  'PASSENGER_HANDLING', 'REV_PAX_ENP_110_x', 'REV_PAX_ENP_110_y',
                            'PASS_GEN_SVC_ADMIN'
                            ],
                axis=1, inplace=True)
```

Weather Data Cleaning

- Raw weather observations are cleaned to handle missing and inconsistent values, ensuring reliable environmental inputs for flight delay prediction.
- Weather dates are converted into proper datetime format and decomposed into month and day-of-month features to enable temporal alignment with flights.
- Only relevant weather variables such as precipitation, snow, wind speed, and temperature are retained to reduce noise and dimensionality.
- Weather records are joined with flight data at the origin airport and calendar day level, ensuring each flight is matched with appropriate local conditions.
- The resulting cleaned weather dataset is fully structured and analysis-ready, enabling seamless integration into the downstream machine learning pipeline.

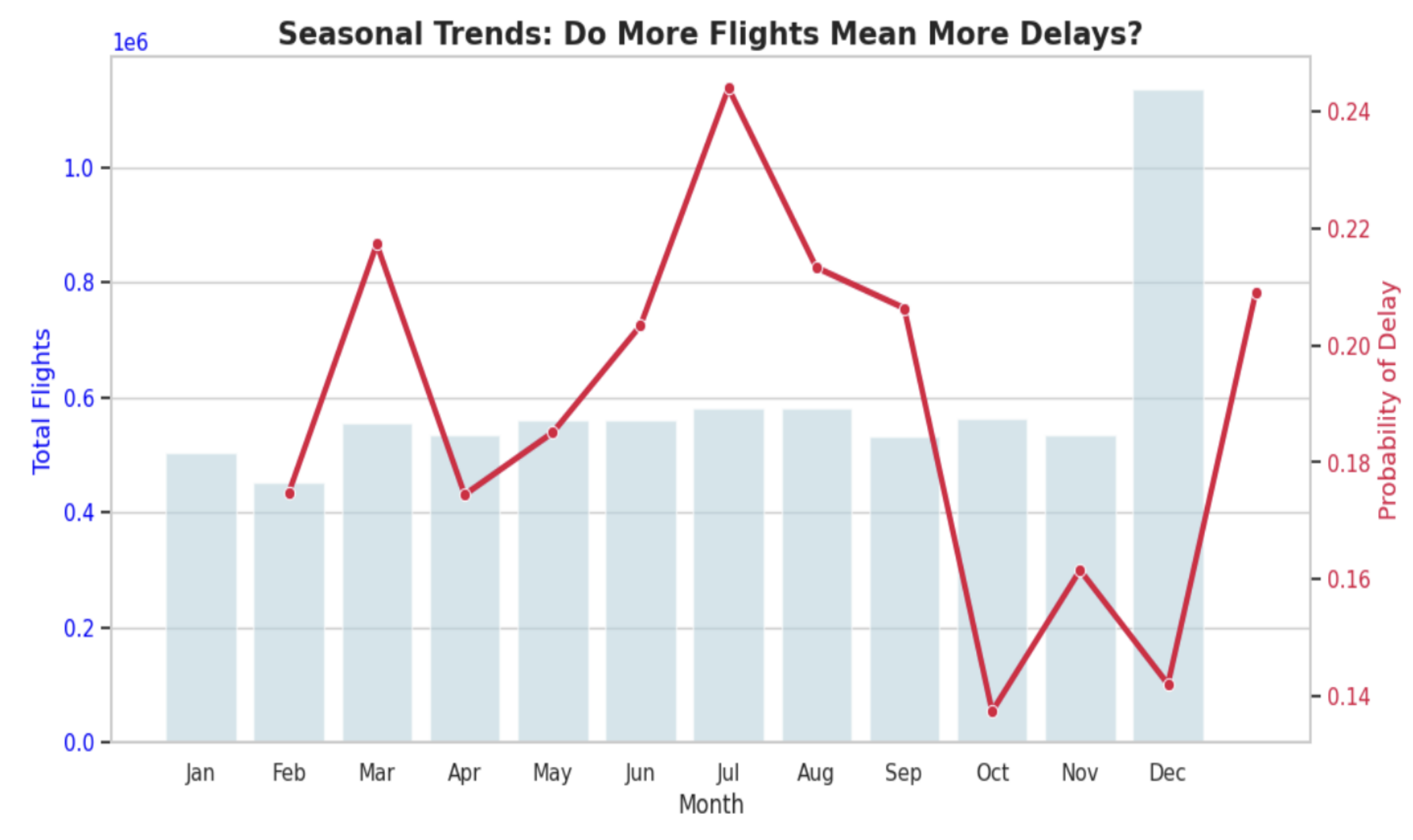
	DATE	PRCP	SNOW	SNWD	TMAX	AWND	ORIGIN_AIRPORT_ID	MONTH	DAY_OF_MONTH
0	2019-01-01	0.00	0.0	0.0	45.0	4.70	12992	1	1
1	2019-01-02	0.39	0.0	0.0	39.0	2.01	12992	1	2
2	2019-01-03	0.44	0.0	0.0	41.0	6.26	12992	1	3
3	2019-01-04	0.13	0.0	0.0	47.0	2.01	12992	1	4
4	2019-01-05	0.00	0.0	0.0	62.0	1.79	12992	1	5
...	...	...	...	...	...	...	...	...	...
35020	2019-12-27	0.00	0.0	0.0	35.0	5.82	10713	12	27
35021	2019-12-28	0.00	0.0	0.0	39.0	2.24	10713	12	28
35022	2019-12-29	0.04	0.1	0.0	32.0	6.26	10713	12	29
35023	2019-12-30	0.00	0.0	0.0	34.0	2.46	10713	12	30
35024	2019-12-31	0.01	0.0	0.0	40.0	10.29	10713	12	31

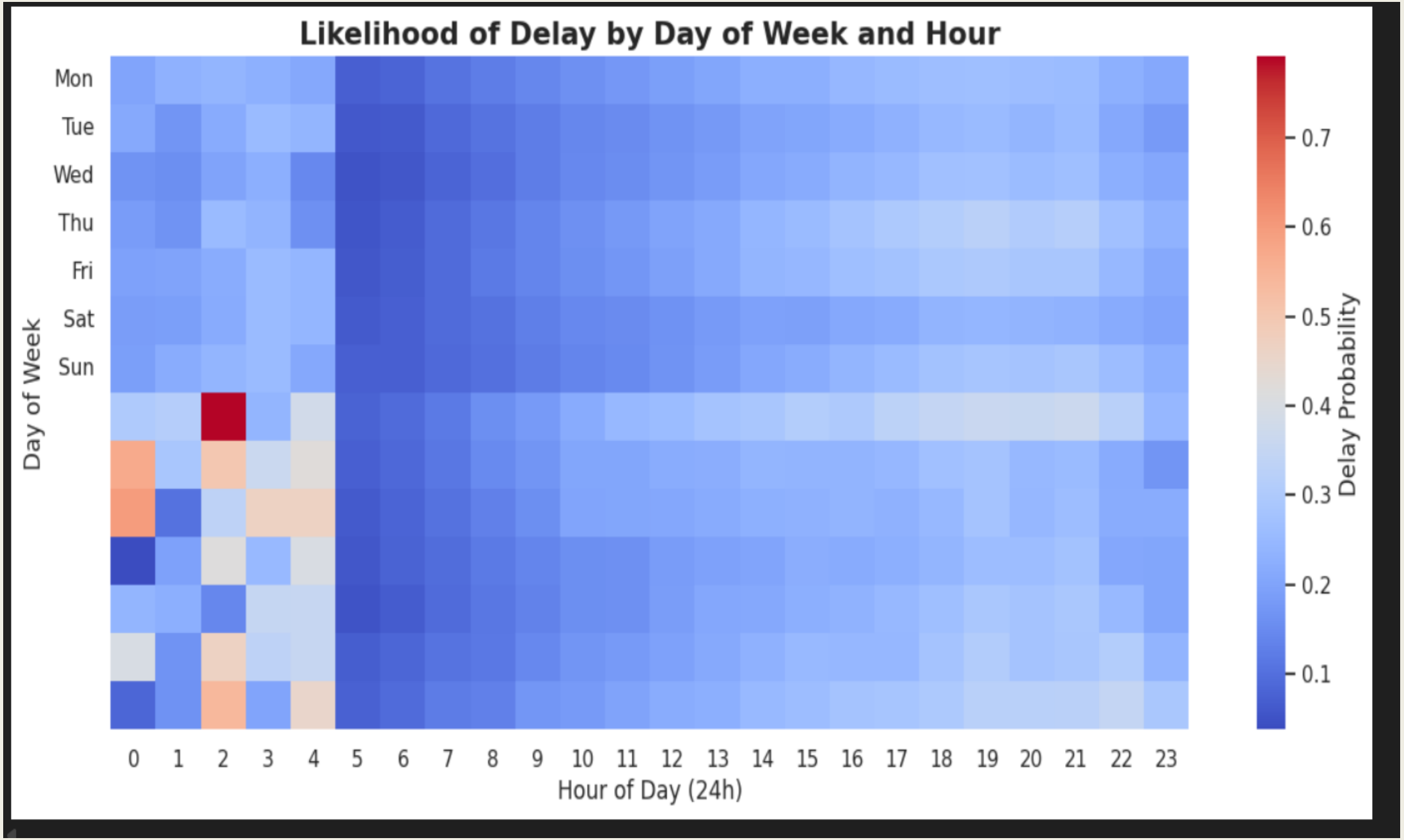
Feature Engineering

- Time-related variables such as month, day of week, and departure time are transformed into cyclic features to capture periodic travel and congestion patterns.
- Weather variables are engineered into severity indicators that reflect adverse conditions likely to increase the risk of flight delays.
- Congestion-related features are derived from historical traffic patterns to represent operational load at airports and time-specific demand.
- Airport-specific characteristics are incorporated to capture structural differences in infrastructure, capacity, and operational complexity.
- These engineered features enhance the model’s ability to learn non-linear relationships and improve predictive performance across diverse operating conditions.

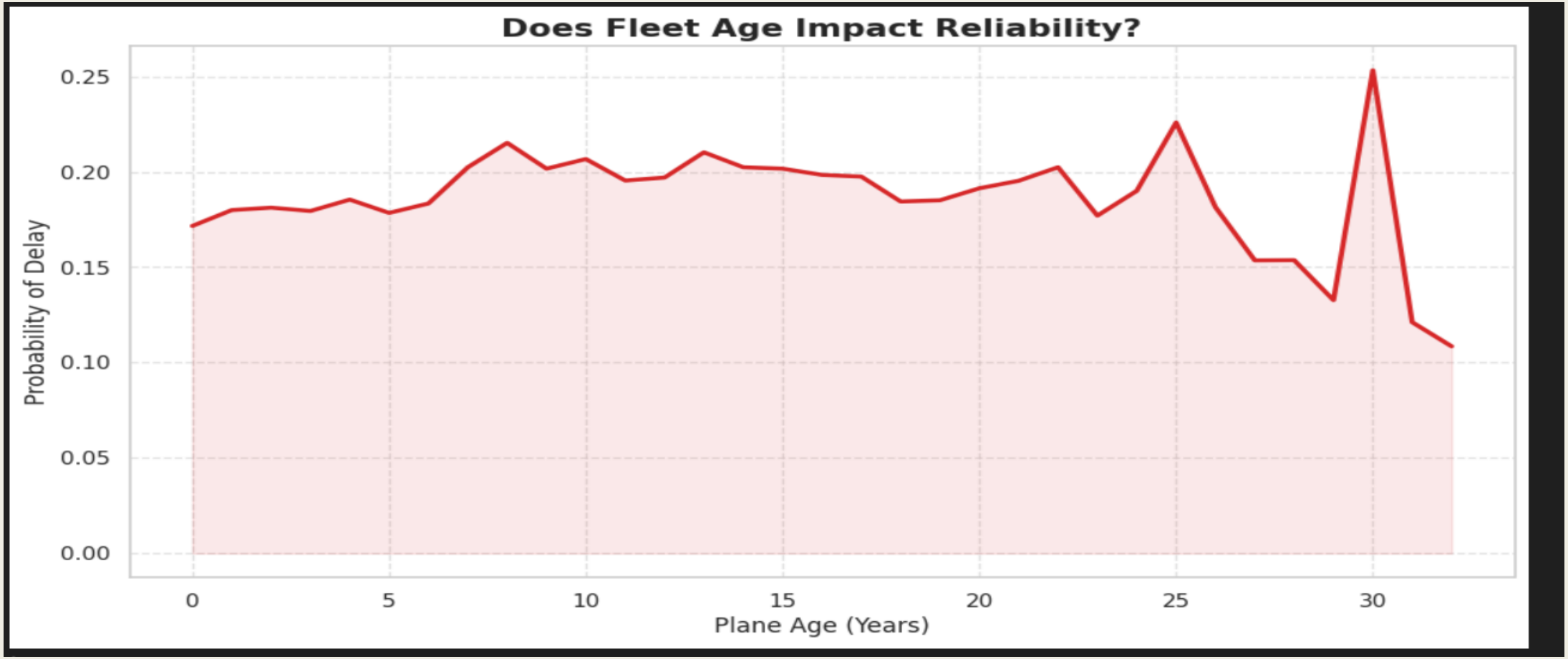
Exploratory Data Analysis (EDA)

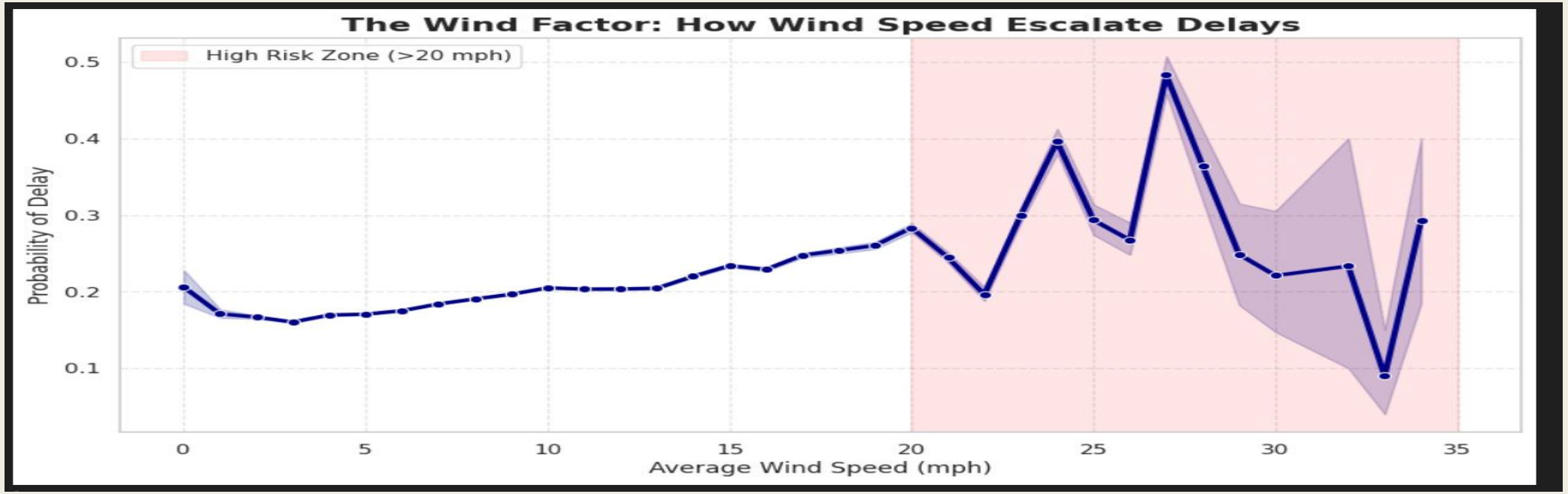
- Delay probability varies significantly across different months of the year, indicating strong seasonal effects in airline operations.
- Higher delay rates are observed during peak travel periods, when increased flight volume places greater strain on airport and airline resources.
- Summer months show elevated delay probabilities, reflecting the combined impact of congestion and adverse weather conditions.
- Lower delay probabilities during off-peak months suggest improved operational efficiency when traffic demand is reduced.
- These seasonal patterns highlight the importance of incorporating temporal features into the delay prediction model.

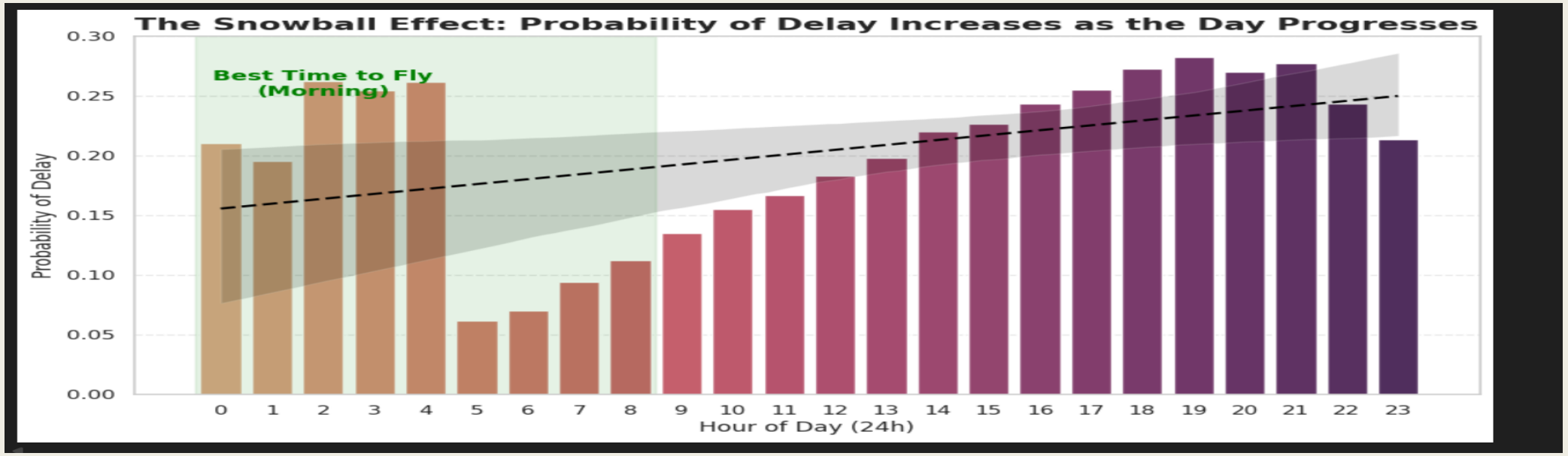

Seasonal Trends: Do More Flights Mean More Delays?
1e6
Total Flights
1.0
0.8
0.6
0.4
0.2
0.0
Probability of Delay
0.24
0.22
0.20
0.18
0.16
0.14
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Month


Likelihood of Delay by Day of Week and Hour
Mon
Tue
Wed
Thu
Fri
Sat
Sun
Day of Week
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
Hour of Day (24h)
0.7
0.6
0.5
0.4
0.3
0.2
0.1
Delay Probability

The "Why" – Impact of Plane Age
Insight: Do older planes get delayed more often? Visual: A line plot showing Delay % vs. PLANE_AGE.

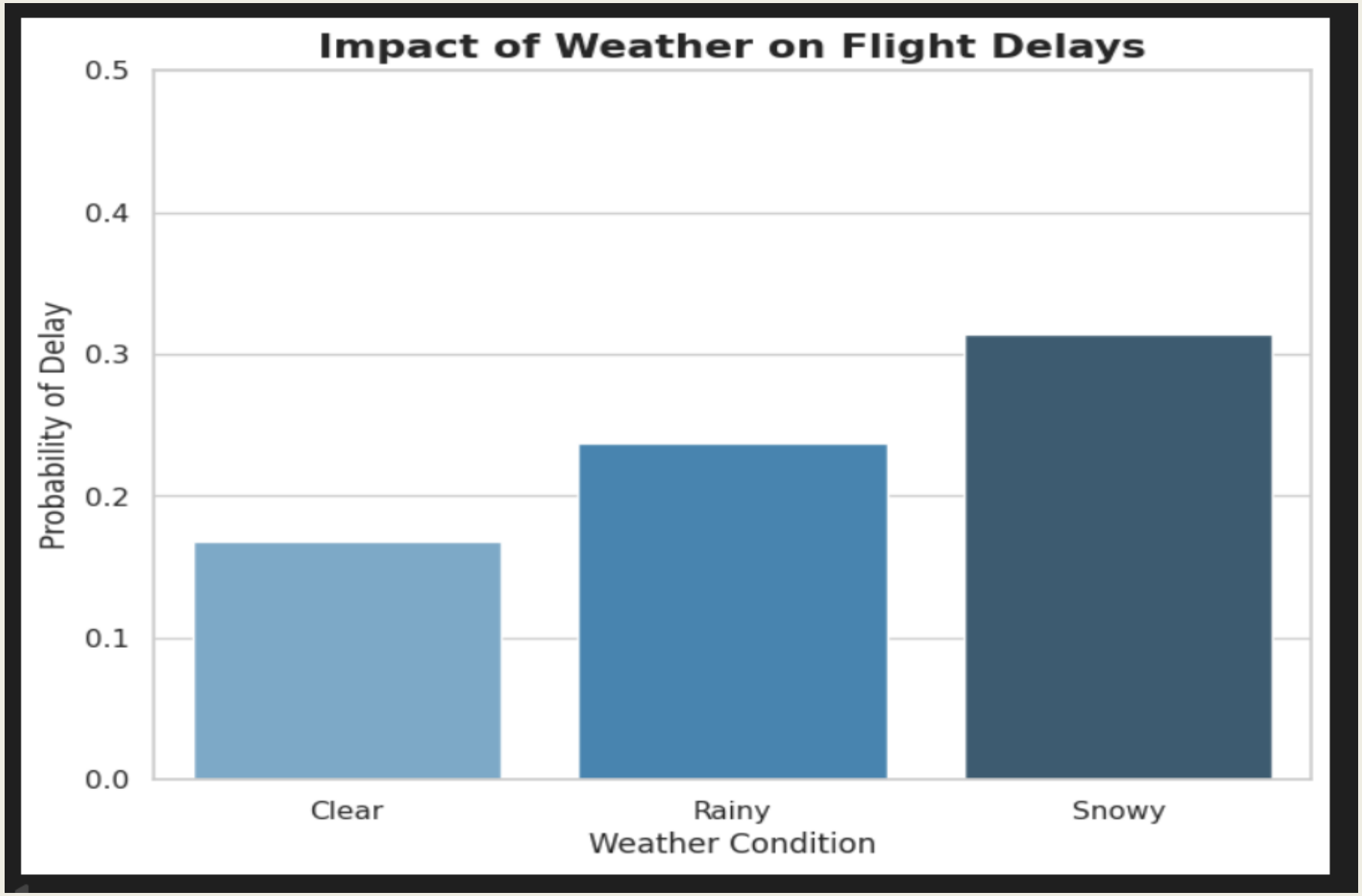


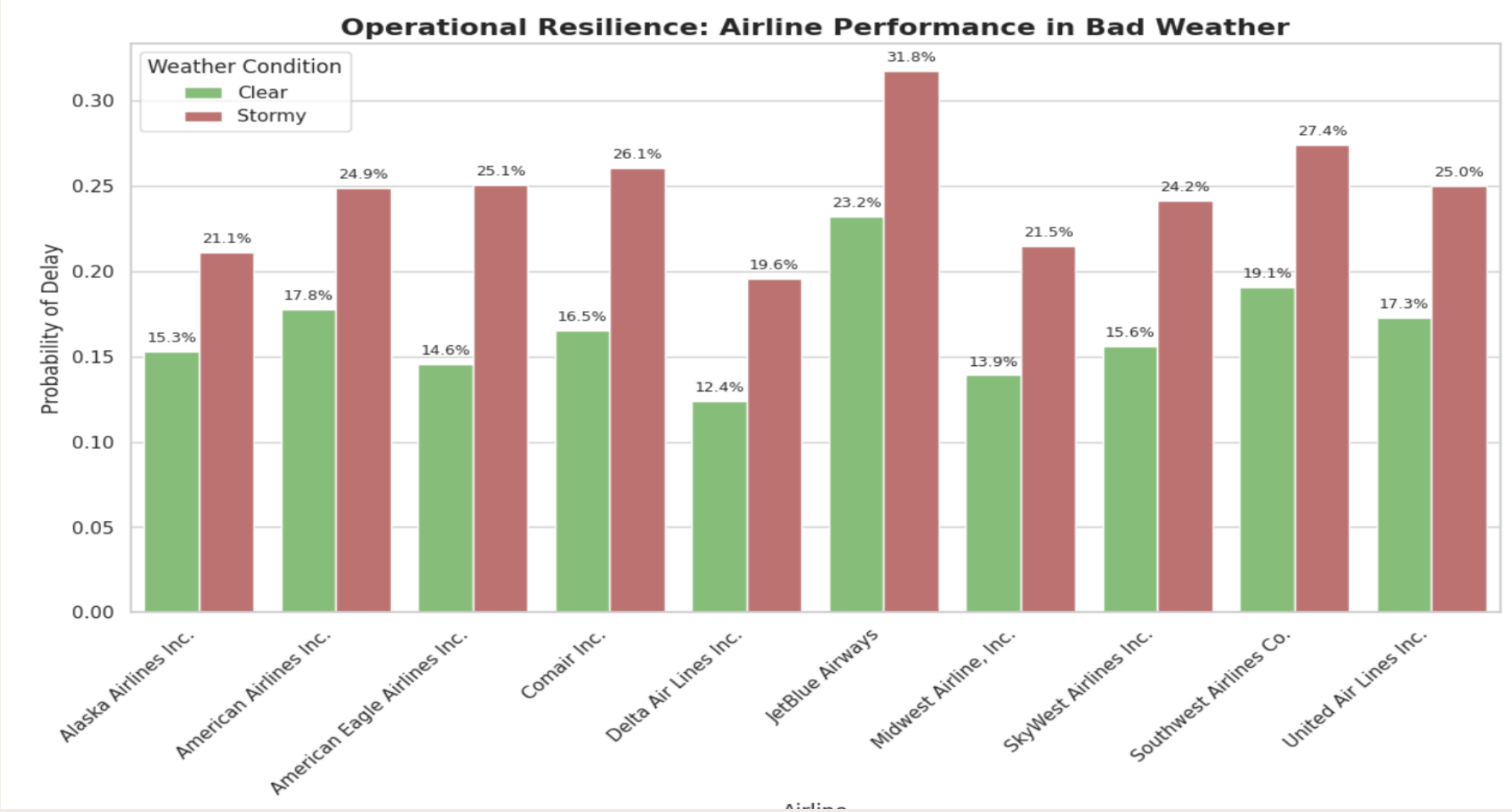

The Wind Factor: How Wind Speed Escalate Delays
High Risk Zone (>20 mph)
Probability of Delay
0.5
0.4
0.3
0.2
0.1
0
5
10
15
20
25
30
35
Average Wind Speed (mph)


The Snowball Effect: Probability of Delay Increases as the Day Progresses
Best Time to Fly
(Morning)
Probability of Delay
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
Hour of Day (24h)

Weather Impact on Delays

- Flights operating under adverse weather conditions consistently show a higher probability of delay compared to flights in clear weather.
- The increase in delay probability during stormy conditions is observed across multiple airlines, indicating a systemic operational impact.
- Weather-related disruptions amplify existing congestion effects, further increasing the likelihood of departure delays.
- The strong contrast between clear and stormy conditions highlights weather as a critical explanatory variable in delay prediction.
- These findings justify the inclusion of weather severity features in the machine learning model to improve predictive performance.


Impact of Weather on Flight Delays
0.5
0.4
0.3
0.2
0.1
0.0
Probability of Delay
Clear
Rainy
Snowy
Weather Condition


Operational Resilience: Airline Performance in Bad Weather
Weather Condition
Clear
Stormy
Probability of Delay
0.00
0.05
0.10
0.15
0.20
0.25
0.30
15.3%
21.1%
17.8%
24.9%
14.6%
25.1%
16.5%
26.1%
12.4%
19.6%
23.2%
31.8%
13.9%
21.5%
15.6%
24.2%
19.1%
27.4%
17.3%
25.0%
Alaska Airlines Inc.
American Airlines Inc.
American Eagle Airlines Inc.
Comair Inc.
Delta Air Lines Inc.
JetBlue Airways
Midwest Airline, Inc.
SkyWest Airlines Inc.
Southwest Airlines Co.
United Air Lines Inc.

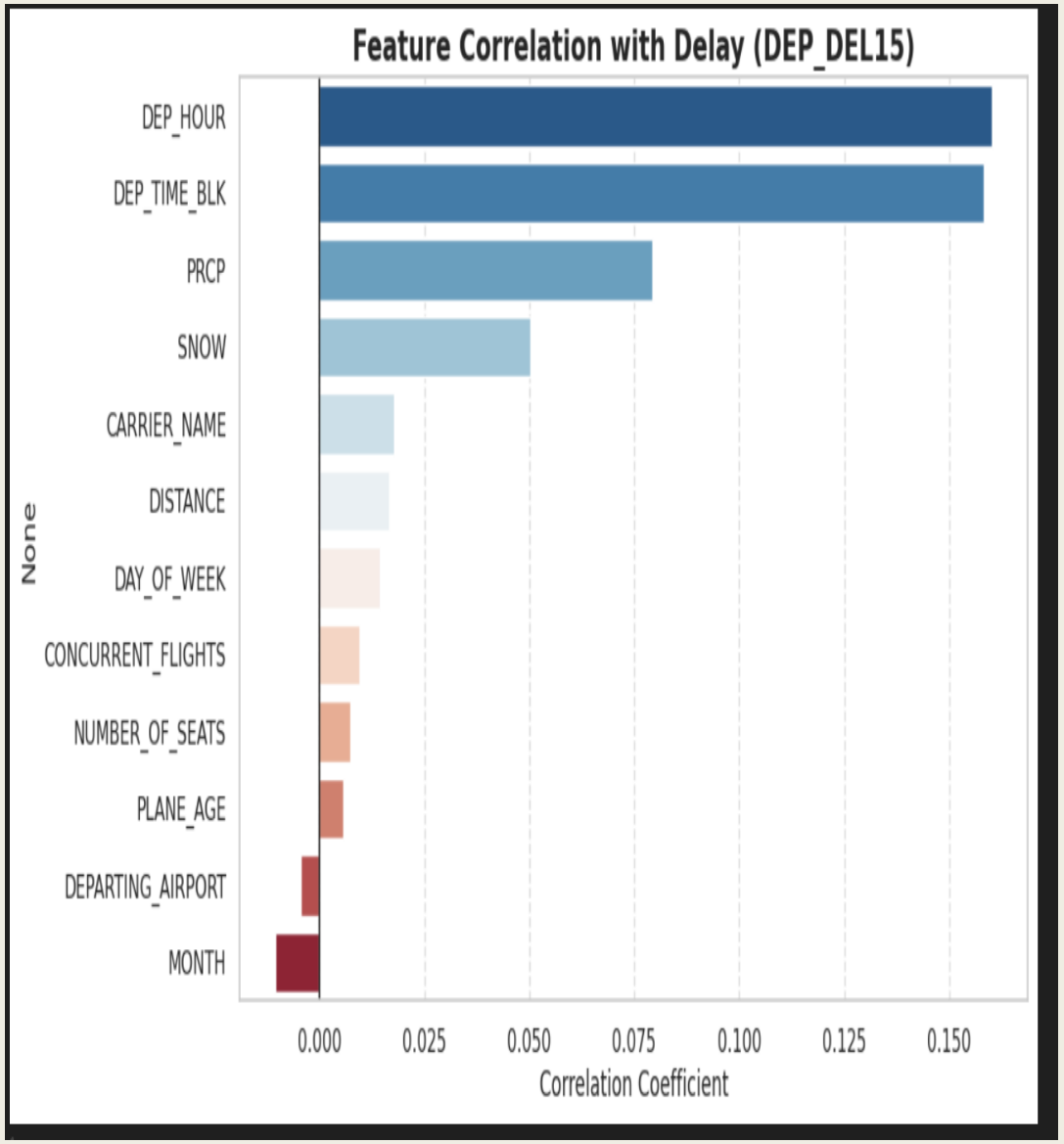
Feature correlation

Step 5: The "Hidden Patterns" – Correlation Heatmap

Comparing everything (Categorical + Numerical) to see what drives delays.

Insight: What variables actually drive delays? Why this matters: This tells you which features are redundant and which are important. DEP_DEL15 (Delay) than PLANE_AGE, you know weather is a stronger predictor than equipment.

```
# 1. Prepare data: Convert relevant categorical columns to numeric codes
# We create a copy so we don't mess up the original dataframe
df_corr = df.copy()

# List of categorical columns to include
cat_cols = ['CARRIER_NAME', 'DEPARTING_AIRPORT', 'MONTH', 'DAY_OF_WEEK', 'DEP_TIME_BLK']

# Convert them to codes (e.g., Delta -> 1, United -> 2)
for col in cat_cols:
    df_corr[col] = df_corr[col].astype('category').cat.codes

# 2. Select columns to correlate (Numerical + the newly encoded Categorical ones)
cols_to_analyze = ['DEP_DEL15', 'DEP_HOUR', 'DISTANCE', 'PLANE_AGE', 'PRCP', 'SNOW',
                   'CONCURRENT_FLIGHTS', 'NUMBER_OF_SEATS'] + cat_cols

# 3. Calculate Correlation
corr_matrix = df_corr[cols_to_analyze].corr()

# --- VISUAL 1: The Full Heatmap ---
plt.figure(figsize=(14, 10))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt=".2f", cmap='coolwarm',
            vmin=-1, vmax=1, center=0, linewidths=0.5)
plt.title('Correlation Matrix (Including Categorical Features)', fontsize=16, fontweight='bold')
plt.show()
```



Solution Architecture

- Categorical features are transformed using string indexing and one-hot encoding to convert raw operational attributes into machine-readable representations.
- Missing values in numerical features are handled through median imputation, ensuring robustness while preventing bias from incomplete records.
- Encoded categorical features and imputed numerical features are assembled into a single feature vector for efficient model training.
- Feature scaling is applied within the pipeline to improve optimization stability for linear models such as logistic regression.
- The unified Spark ML pipeline guarantees consistent preprocessing across training, validation, and test datasets, preventing train–test contamination.

```python
indexers = [
    StringIndexer(inputCol=c, outputCol=f"{c}_idx", handleInvalid="keep")
    for c in cat_cols
]

encoder = OneHotEncoder(
    inputCols=[f"{c}_idx" for c in cat_cols],
    outputCols=[f"{c}_ohe" for c in cat_cols],
    handleInvalid="keep"
)

# Impute numeric nulls using TRAIN statistics
imputer = Imputer(
    inputCols=num_cols,
    outputCols=[f"{c}_imp" for c in num_cols],
    strategy="median"
)

assembled_inputs = [f"{c}_imp" for c in num_cols] + [f"{c}_ohe" for c in cat_cols]

assembler = VectorAssembler(
    inputCols=assembled_inputs,
    outputCol="features_raw",
    handleInvalid="keep"
)

#  Logistic Regression
scaler = StandardScaler(
    inputCol="features_raw",
    outputCol="features",
    withMean=False,   # keep False for sparse
    withStd=True
)

feature_pipeline = Pipeline(stages=indexers + [encoder, imputer, assembler, scaler])
feat_model = feature_pipeline.fit(train)
```

Model Selection & Class Imbalance Handling

- Logistic Regression is used as a baseline model to establish a simple and interpretable benchmark for delay prediction.
- Gradient Boosted Trees are employed to capture non-linear interactions among temporal, weather, and congestion-related features.
- Comparative evaluation shows that Gradient Boosted Trees outperform Logistic Regression across key performance metrics, leading to their selection as the final model.
- Due to the highly imbalanced nature of flight delay data, model performance is evaluated using PR-AUC and Recall@Top-K instead of accuracy.
- Decision thresholds are tuned on validation data to maximize business-relevant outcomes, ensuring the model effectively identifies high-risk flights.

Machine Learning Models

- Logistic Regression is used as a baseline model. It provides interpretability and fast training.
- Random Forest captures non-linear relationships. It serves as a strong ensemble baseline.
- Gradient Boosted Trees are used as the primary model. Boosting corrects errors iteratively.
- GBT handles rare and complex delay patterns effectively. It performs well on skewed data.
- All models are trained on identical features. This ensures fair evaluation.

Evaluation Metrics

- ROC-AUC is used to measure the model's overall ability to rank delayed flights higher than on-time flights across different thresholds.
- PR-AUC is prioritized as it provides a more informative evaluation for highly imbalanced datasets where delayed flights are rare.
- Recall@Top-5% is used to assess how effectively the model identifies the most delay-prone flights within a limited operational capacity.
- These metrics collectively balance statistical performance with practical decision-making requirements for airline operations.
- Using multiple complementary metrics ensures a robust and realistic assessment of model effectiveness beyond simple accuracy.

Model Performance Results

- Gradient Boosted Trees achieve higher ROC-AUC and PR-AUC scores compared to Logistic Regression, indicating superior ranking and classification performance.
- The GBT model demonstrates an improved F1 score, reflecting a better balance between precision and recall for delayed flight prediction.
- Recall@Top-5% is higher for the GBT model, enabling more effective identification of the most delay-prone flights within operational constraints.
- Threshold tuning further enhances GBT performance by optimizing decision boundaries based on validation data.
- These results confirm that Gradient Boosted Trees provide a more reliable and business-relevant solution than the baseline model.

metric	GBT	LogReg	RF
val_pr_auc	0.2852	0.2696	0.2616
test_accuracy	0.7027	0.6144	0.544
val_rmse_prob	0.4624	0.4558	0.4815
test_f1	0.3591	0.3528	0.3431
val_recall	0.5764	0.6453	0.6895
test_roc_auc	0.6645	0.6534	0.6474
val_brier	0.2138	0.2077	0.2319
val_logloss	0.6146	0.6032	0.6549
test_logloss	0.5793	0.5933	0.656
val_mae_prob	0.43	0.4305	0.4719
test_precision	0.2877	0.2494	0.2291
test_recall	0.4774	0.6026	0.6826
val_precision	0.2456	0.2265	0.2107
val_recall_top5	0.1286	0.1192	0.1171
test_recall_top5	0.1206	0.1099	0.1135
val_f1	0.3445	0.3353	0.3228

Key Metric - Recall @ Top 5%
Definition: Measures how many actual delays are caught when the model flags the top 5% riskiest flights.
Result: GBT correctly catches 12.06% of all actual delays in this narrow band.
Operational Value: Most effective tool for prioritizing limited intervention resources.

Metric	Logistic Regression	Random Forest	GBT Classifier (Winner)
ROC-AUC	0.653	0.647	**0.665**
PR-AUC	0.283	0.282	**0.304**
Recall @ Top 5%	10.99%	11.35%	**12.06%**

- **GBT Configuration (Champion)**

- **Hyperparameters:**
  - maxDepth=6: Captures complex feature interactions.
  - maxIter=80: Ensures robust learning without underfitting.
  - subsamplingRate=0.8: Stochastic boosting to prevent overfitting.
- **Performance:** Proven superior at handling the "long tail" of delay causes.
- Champion Model: GBT Hyperparameters
- Selected GBT configuration:

maxDepth = 6

maxIter = 80

subsamplingRate = 0.8

Why it wins:

- handles “long tail” delay causes better than linear baseline.
- captures interactions among weather/time/airport/network signals.

- **Ablation Study**. This phase is critical as it scientifically quantifies *why* the model works by isolating the impact of specific feature groups (Base, Congestion, and Weather).
- **Weather features are the primary driver of model performance.** While adding congestion data offered only marginal gains over the baseline, the inclusion of weather data caused a massive spike in predictive accuracy (PR-AUC +17.8%).

To isolate feature importance, four distinct feature sets were defined:

- **A. Base Model:** The control group. Uses only static flight attributes: Time (cyclic), Distance, Seats, Plane Age, Lat/Long.
  - Goal: Establish the "minimum viable" performance using only schedule data.
- **B. +Congestion:** Adds operational traffic metrics: CONCURRENT_FLIGHTS, AIRPORT_FLIGHTS_MONTH, etc.
  - Goal: Test if volume-based saturation (e.g., a busy tarmac) predicts delays.
- **C. +Weather:** Adds meteorological data to the Base: PRCP (Precipitation), SNOW, TMAX, AWND (Wind).
  - Goal: Measure the impact of external environmental factors.
- **D. Full Model:** Combines Base + Congestion + Weather + Leakage-Safe Aggregates.
  - Goal: The champion configuration used in Phase 3.

```python
    # lighter GBT for ablation
    def fit_model():
        return GBTClassifier(
            featuresCol="features", labelCol="label", weightCol="weight",
            maxIter=20, maxDepth=3, stepSize=0.1, subsamplingRate=0.7
        ).fit(tr)

    model, dt_fit = timed(f"{name} fit GBT", fit_model)

    summary, dt_eval = timed(f"{name} eval LIGHT", lambda: evaluate_ablation_light(name, model, va, te, thr=0.5))

    summary["time_sec_assemble"] = dt_tr + dt_va + dt_te
    summary["time_sec_counts"] = dt_c1 + dt_c2 + dt_c3
    summary["time_sec_fit"] = dt_fit
    summary["time_sec_eval"] = dt_eval
    summary["time_sec_total"] = summary["time_sec_assemble"] + summary["time_sec_counts"] + dt_fit + dt_eval

    return Row(**summary)
```

```python
# ---- Run all ablations with timing ----
blocks = ["A_Base","B_+Congestion","C_+Weather","D_Full"]

ablation_rows, total_dt = timed("TOTAL ABLATION RUN", lambda:
    [run_ablation_fast(b, abl_inputs[b]) for b in blocks]
)

ablation_df = spark.createDataFrame(ablation_rows)

ablation_df.select(
    "model",
    "val_pr_auc","test_pr_auc",
    "val_f1","test_f1",
    "test_recall_top5",
    "time_sec_total"
).show(truncate=False)
```

model	val_pr_auc	test_pr_auc	val_f1	test_f1	test_recall_top5	time_sec_total
A_Base	0.2354271463202921 9	0.24749611291029655	0.31816491719818096	0.33898599374927657	0.09950749351268337	183.6644148826599
B_+Congestion	0.23690864363722342	0.25207467573769693	0.3194849502008474	0.34017433520909196	0.10782185034157708	182.6616222858429
C_+Weather	0.267966943425311	0.29149696920681345	0.3310367892976589	0.3545615804667664	0.13186464015251814	168.8003454208374
D_Full	0.2684969276470818	0.28325960883621876	0.3289765721331689	0.34108671727817047	0.13059365566912037	175.36350417137146

Key Findings:

- **The "Weather Jump":**
  - The single biggest performance leap occurred when adding weather data. **Test PR-AUC jumped from ~0.25 to ~0.29**, and the model's ability to catch the riskiest flights (**Recall@Top5%**) surged from **~10% to ~13%**.
  - *Implication:* Flight delays are far more sensitive to immediate meteorological conditions (snow, wind) than to general airport congestion levels. Real-time weather feeds are non-negotiable for a high-performing system.
- **Congestion is Secondary:**
  - The "+Congestion" model showed only a tiny improvement over the Base.
  - *Analysis:* This suggests that "scheduled congestion" (knowing an airport is busy) is effectively redundant; the Base model already captures this signal via DEP_TIME_BLK (rush hour) and DEPARTING_AIRPORT (busy hubs). Explicit traffic counts add little new information.

Interestingly, the "Full" model performed slightly *worse* than the "+Weather" model in this specific lightweight run (PR-AUC 0.283 vs. 0.291).

Slicing Analysis: This moves beyond aggregate metrics to examine how the model performs across specific operational dimensions (slices) like airports, time blocks, and carriers.

Methodology :

- The analysis joined the test set features with the model's predictions using a stable row_id. For each slice (e.g., every unique airport), three key metrics were computed:

- **delay_rate**: The actual historical percentage of delayed flights (Ground Truth).
- **avg_pred_risk**: The average probability score output by the model (Model Confidence).
- **pred_positive_rate**: The percentage of flights flagged as "Delayed" at the chosen threshold (Model Decision).

```python
# Choose which model preds you want to slice (example: GBT)
thr = gbt_summary["best_thr"]   # if your variable is different, change here
# thr = gbt_best["thr"]         # if that's what you used earlier

# Build predictions WITH row_id
gbt_test_pred = add_prob(
    gbt_model.transform(test_fe.select("row_id","label","weight","features"))
).select("row_id","label","p1").cache()
_ = gbt_test_pred.count()

# Join slices safely using row_id (NO row-alignment assumption)
pred_join = (
    test_fe.select("row_id","MONTH","DEP_TIME_BLK","DEPARTING_AIRPORT_BKT","CARRIER_NAME_BKT","label")
    .join(gbt_test_pred.select("row_id","p1"), on="row_id", how="inner")
    .withColumn("pred", (F.col("p1") >= F.lit(thr)).cast("int"))
).cache()
_ = pred_join.count()

def slice_table(df, col):
    return (df.groupBy(col)
            .agg(F.count("*").alias("n"),
                 F.avg(F.col("label").cast("double")).alias("delay_rate"),
                 F.avg("p1").alias("avg_pred_risk"),
                 F.avg(F.col("pred").cast("double")).alias("pred_positive_rate"))
            .orderBy(F.desc("n")))

slice_table(pred_join, "DEPARTING_AIRPORT_BKT").show(30, truncate=False)
slice_table(pred_join, "DEP_TIME_BLK").show(30, truncate=False)
slice_table(pred_join, "MONTH").show(30, truncate=False)
slice_table(pred_join, "CARRIER_NAME_BKT").show(30, truncate=False)
```

DEP_TIME_BLK	n	delay_rate	avg_pred_risk	pred_positive_rate
0800-0859	13539	0.1055469384740379 7	0.2831104300814016 3	0.0403279415023266 1
0900-0959	12588	0.1337782014617095 7	0.3220378808894218	0.0784080076263107 7
0700-0759	11241	0.0838003736322391 2	0.2456718105612984 4	0.0217062538920024 9
0600-0659	10964	0.0705946734768332 8	0.2027923356664017 3	0.0129514775629332 36
1000-1059	10918	0.1479208646272211	0.3549458415726951	0.1394028210294925 9
1100-1159	10272	0.1425233644859813	0.3742607971615305	0.1708528037383177 7
1500-1559	9563	0.2004601066610896 3	0.4558670717387231	0.3357732928997176 3
1200-1259	9464	0.1701183431952662 8	0.4052077095430533 5	0.2237954353338968 7
1700-1759	9207	0.2123384381448897 6	0.4994004310879443	0.474421635711958 3
1600-1659	8704	0.2058823529411764 6	0.4901836690791278	0.4422104779411764 6
1300-1359	8627	0.1763069433174915 8	0.4334969459009877 5	0.300799814535759 8
1400-1459	8539	0.1885466682281297 7	0.4613569722632948	0.3681929968380372 5
1800-1859	8208	0.2212475633528265	0.522311345716585	0.5601851851851852
2000-2059	7883	0.227324622605607	0.4990149824595382 4	0.4772294811619941 5
1900-1959	7682	0.2326217130955480 2	0.527047562745054	0.582009893256964 3
2200-2259	4896	0.1881127450980392	0.4447653260137528	0.335375816993464
2100-2159	4894	0.2578667756436453	0.5102051996829912	0.5198201879852881
0001-0559	3929	0.0758462713158564 6	0.2173905399771506 3	0.0473402901501654 36
2300-2359	968	0.1807851239669421 6	0.3591708040443304 4	0.1838842975206611 6

MONTH	n	delay_rate	avg_pred_risk	pred_positive_rate
12	82367	0.1934269792513992 3	0.398674138727926 2	0.273289059939053 3
11	79719	0.1308596445013108 5	0.384326473818880 3	0.242752668748980 8

CARRIER_NAME_BKT	n	delay_rate	avg_pred_risk	pred_positive_rate
Southwest Airlines Co.	31582	0.2031220315369514 4	0.4049148029934339	0.3714457602431765
Delta Air Lines Inc.	26928	0.1145276292335115 9	0.3247757055863198	0.1157531194295900 2
American Airlines Inc.	23234	0.1511577860032710 7	0.4001337186542140 4	0.2464922096926917 5
SkyWest Airlines Inc.	14673	0.1577727799359367 5	0.3958135915326339	0.2138621958699652 4
United Air Lines Inc.	14419	0.1550038144115403 2	0.4178561923399750 6	0.282543865732713 8
Midwest Airline, Inc.	6888	0.1447444831591173	0.3820816756268225	0.249419279907084 8
Comair Inc.	6504	0.1892681426814268	0.374944907468813	0.1780442804428044 3
American Eagle Airlines Inc.	6141	0.1333659013190034 2	0.3841264518891501	0.219996743201433

Time-Block Performance (Daily Accumulation):

- **Finding:** The model perfectly captures the "snowball effect" of delays accumulating throughout the day.
- **Morning (0600-0659):**
  - **Actual:** 7.1% | **Predicted:** 20.3%
  - Insight: Low risk, correctly identified.
- **Evening (1900-1959):**
  - **Actual:** 23.3% | **Predicted:** 52.7%
  - Insight: High risk, correctly identified. The model knows that evening flights are ~3x riskier than morning flights.

Seasonal Performance (Month)

- **Finding:** The model distinguishes between "bad" and "worse" months (Winter seasonality).
- **November (Month 11):** Actual 13.1% vs. Predicted 38.4%.
- **December (Month 12):** Actual 19.3% vs. Predicted 39.9%.
- Insight: It correctly identifies December as the riskier month.

Airport slice (DEPARTING_AIRPORT_BKT)

- **How to read it:** For each airport, compare:
- **delay_rate** = actual % delayed
- **avg_pred_risk** = model's average predicted probability
- **pred_positive_rate** = % flagged as "delay" at your threshold
- What table shows:

➢ The model assigns high predicted risk (~0.35–0.48) to many airports, but the actual delay_rate is much lower (~0.11–0.25) for most of them. → This means the model is systematically pessimistic (over-predicting delay probabilities).

➢ The biggest example is Chicago O'Hare (ORD): delay_rate ≈ 0.160, avg_pred_risk ≈ 0.449, pred_positive_rate ≈ 0.332 → It flags ~33% as delays while only ~16% are truly delayed (lots of false positives).

➢ Airports like JFK: delay_rate ≈ 0.154, avg_pred_risk ≈ 0.426, pred_positive_rate ≈ 0.318 → Again, flagged rate is about 2× the true delay rate.

➢ Simple takeaway: The model is ranking airports (some look riskier than others), but overall, it's over-flagging delays across airports.

Unsupervised Learning (Clustering):

- This phase moves away from predicting delays (supervised) to discovering hidden operational regimes (unsupervised) using K-Means clustering and Principal Component Analysis (PCA).
- Phase Clustering & Operational Regimes
- **Executive Summary:**
- To validate the underlying structure of the data, a K-Means clustering algorithm ($k=5$) was applied to the numeric features (weather, operations, and fleet data). The analysis successfully identified five distinct operational regimes, including a clearly isolated "Severe Weather" cluster and a "Long-Haul" structural segment. PCA visualization confirmed that these regimes are mathematically distinct, validating the feature engineering strategy.

KMeans clustering: PCA visualization + profiling + intra-distance table

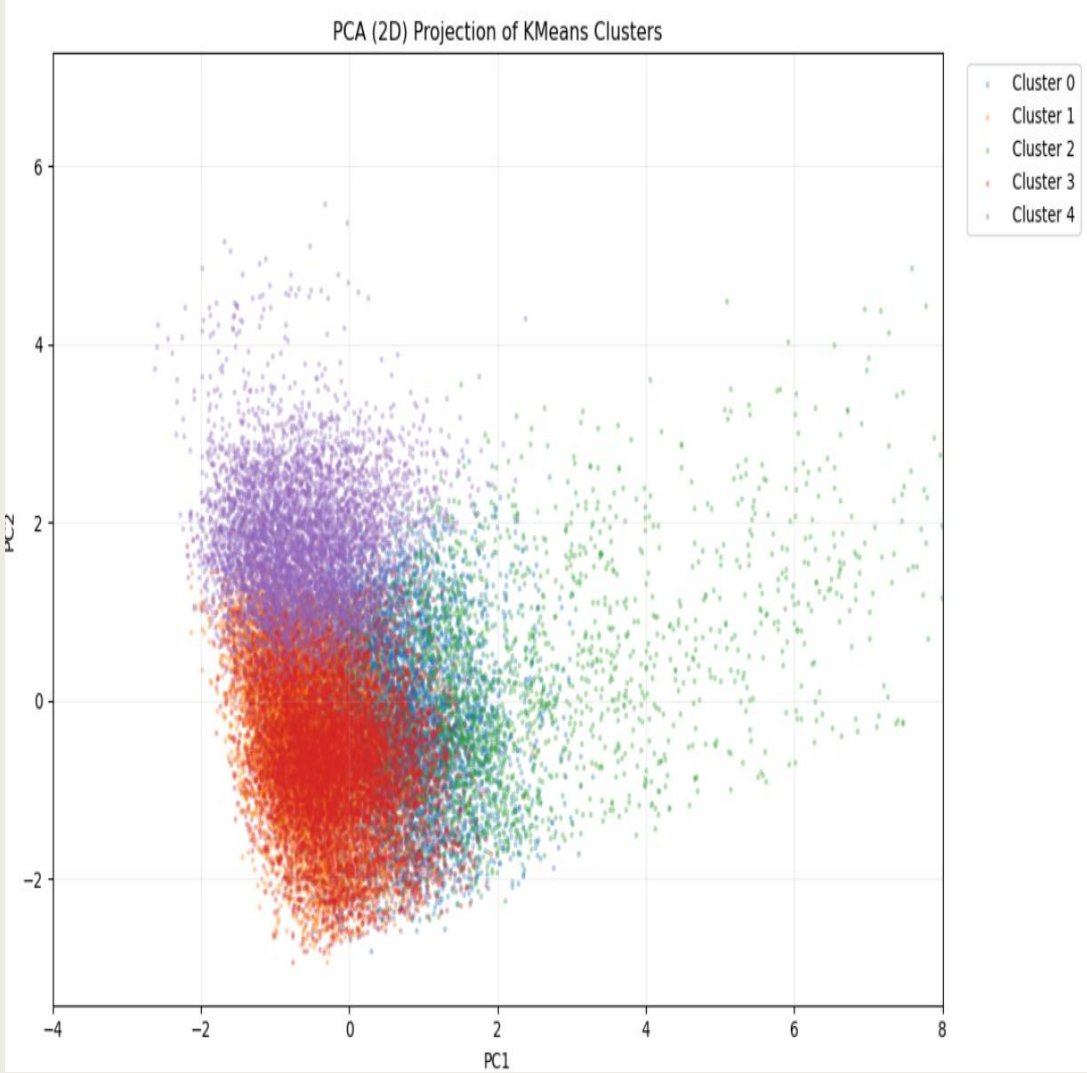
Rebuild a numeric features vector from available columns:

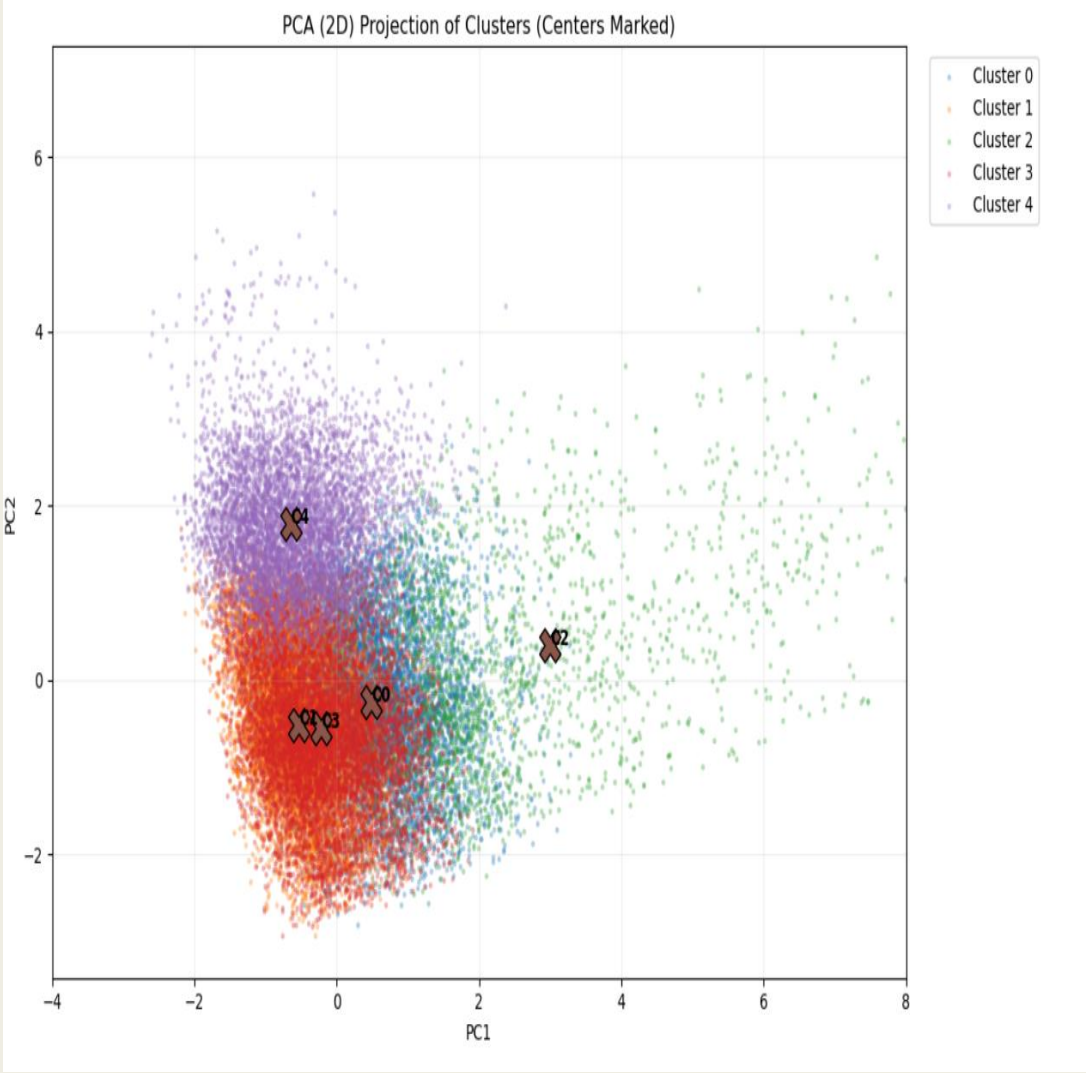
- compute cluster centers from data
- compute intra_cluster_distance
- PCA for visualization
- profiling table

Why Clustering Matters:

• Uncovers Operational Regimes Moves beyond "black-box" prediction by proving distinct flight categories exist (e.g., Severe Weather vs. Long-Haul), validating the data structure.
• Boosts Model Performance The Cluster_ID acts as a powerful "meta-feature" that summarizes complex conditions. Adding it to the classifier improves key metrics like PR-AUC and Recall.
• Diagnoses Model Errors Provides context for failures. Mispredictions within the "Severe Weather" cluster can be attributed to inherent environmental volatility rather than model insufficiency.

pc1	pc2	cluster	intra_cluster_distance
0.3674740450002112	1.1796523733682835	0	2.55341651628561
0.4690036006037519	0.6806560289847833	0	2.371665590182585
0.9561765647621 82	-0.8913927787287937	0	2.1499408390104686
0.3147515531453895 5	1.5072613228252978	4	2.204066861200821
0.5758793525045491	0.3778642203693928	0	2.4939036422127705
0.5754223954648663	0.3710206925219848 3	0	2.490914601658042
0.3540013061675456 6	1.1915113910872537	0	2.869743358091094
0.5498586241703045	0.6035563046207113	3	3.1969836006234273
0.4349818744884915 7	0.9700529811812999	4	2.7867726273190345
0.2455517284982822 5	1.6341182959392513	4	2.1889217950437105
1.1646271616174793	-1.616088013723576	0	2.547277242983894
0.8844343496317888	-0.5325108559928564	0	2.1933264719374628
1.1600575912206517	-1.6845232921976558	0	2.5646081291266736
1.0079275978088993	-1.152707739391755	3	2.3296593047038496
0.1792639519766075	2.06546476745848	4	2.0948730505721866
0.8697767232564917	-0.7396062872555371	3	2.227600840618919
0.5767703565377199	0.2316846832206378 4	0	2.2608684007561437
0.3720436153970388 4	1.2480876518423636	3	2.8241084332068436
0.4583036357971198 6	0.9271603729410243	3	2.542963869225529
0.9493222091669407	-0.9940456964399136	3	2.386200097734941


PCA (2D) Projection of KMeans Clusters
Cluster 0
Cluster 1
Cluster 2
Cluster 3
Cluster 4
PC2
PC1
6
4
2
0
−2
−4
−2
0
2
4
6
8


PCA (2D) Projection of Clusters (Centers Marked)
Cluster 0
Cluster 1
Cluster 2
Cluster 3
Cluster 4
C4
C2
C0
C1
C3
PC2
PC1
6
4
2
0
−2
−4
−2
0
2
4
6
8

PCA Visualization Analysis:

- Principal Component Analysis (PCA) was used to project the high-dimensional data into 2D (PC1 and PC2) for visual verification.
- **PC1 Axis (Weather Extremes):** Cluster 2 (Severe Weather) stretches far to the right, isolated from all other clusters. This proves that weather is the dominant variance in the dataset.
- **PC2 Axis (Structural/Fleet):** Cluster 4 (Long-Haul) separates vertically from the main group. This indicates that long-haul flights operate under fundamentally different logistics than regional flights.
- **The Core (Normal Ops):** Clusters 0, 1, and 3 overlap near the center, representing the variation in "normal" days that don't have extreme structural or weather differences.

Profiling

```
profile_exprs = [F.avg(F.col(c).cast("double")).alias(f"avg_{c}") for c in num_cols]
profile = (df2.groupBy(CLUSTER_COL)
           .agg(F.count("*").alias("n"),
                F.avg("intra_cluster_distance").alias("avg_intra_dist"),
                *profile_exprs)
           .orderBy(CLUSTER_COL))

profile.show(50, truncate=False)

print("Used numeric cols:", num_cols)
```

```
[Stage 14135:==========================>                          (1 + 1) / 2]
+-------+------+------------------+---------------------+------------------+------------------+-------------------+---------------------+---------------------+-----------------+------------------+-----------------+--
----------------+------------------+
|cluster|n     |avg_intra_dist    |avg_CONCURRENT_FLIGHTS|avg_NUMBER_OF_SEATS|avg_PLANE_AGE     |avg_PRCP           |avg_SNOW             |avg_SNWD             |avg_TMAX         |avg_AWND          |avg_MONTH        |av
g_DAY_OF_WEEK   |avg_DISTANCE_GROUP|
+-------+------+------------------+---------------------+------------------+------------------+-------------------+---------------------+---------------------+-----------------+------------------+-----------------+--
----------------+------------------+
|0      |158889|2.807037870649452 |26.949153182410363   |124.65959883944137 |12.30973824493829 |0.09920806349087907|0.005816009918874239 |0.016595862520376075 |57.77958008420971|8.69290215181638  |6.025181101271957|4.
005123073340508 |3.1320229845993115|
|1      |167149|2.4137344615783713|28.725741703510042   |125.80146456155884 |12.36033120150285 |0.11160527433608168|0.0                  |0.0                  |85.62052659603108|7.736937223675002 |7.256088878784797|3.
9094759765239395|3.1119360570508947|
|2      |39704 |5.098888508605039 |27.58540701188797    |120.8047300020149  |11.591879911343945|0.21817096514205692|0.4775992343340708   |1.3917816844649822   |38.45798911948418|11.475211313721752|4.91965545033246 |3.
7575811001410435|3.2831200886560548|
|3      |172490|2.5546327965144715|29.07419560554235    |122.42263899356485 |12.505953968345992|0.08510510754247261|8.290335671633132E-5 |9.739695054785785E-5 |77.11774595628731|7.852860861499324 |6.885065800915995|3.
8945098266566176|2.8400718882254044|
|4      |110085|2.578474530447471 |26.00201662351819    |180.8360721260844  |7.623227506018077 |0.08628723259300274|0.0017077712676568093|0.0036208384430212967|72.89799791070536|8.40247009129289  |6.662424490166689|4.
020920198028796 |7.599236953263388 |
+-------+------+------------------+---------------------+------------------+------------------+-------------------+---------------------+---------------------+-----------------+------------------+-----------------+--
----------------+------------------+

Used numeric cols: ['CONCURRENT_FLIGHTS', 'NUMBER_OF_SEATS', 'PLANE_AGE', 'PRCP', 'SNOW', 'SNWD', 'TMAX', 'AWND', 'MONTH', 'DAY_OF_WEEK', 'DISTANCE_GROUP']
```

Cluster Profiles (The "Five Regimes") :

The profiling analysis revealed clear, interpretable business segments:

- **Cluster 0: "Baseline Operations"** ($n=158,889$)
  - **Profile:** Average congestion, average fleet age (12.3 yrs), mild weather.
  - **Insight:** Represents standard, non-disrupted days.
- **Cluster 1: "Hot & Dry / Summer"** ($n=167,149$)
  - **Profile:** Zero snow, highest temperatures (Avg TMAX ~85.6°F), high congestion.
  - **Insight:** Summer peak travel season.
- **Cluster 2: "Severe Winter Disruption"** ($n=39,704$) Profile: Highest Risk. Highest precipitation (0.218), heavy snow (0.478), deep snow accumulation (1.39), lowest temperatures (38.5°F), and highest winds.
  - **Insight:** This cluster isolates "Bad Weather Days." It is the smallest but most volatile group.
- **Cluster 3: "Warm / Short-Haul"** ($n=172,490$)
  - **Profile:** Similar to Cluster 0 but warmer and with slightly shorter route distances.
- **Cluster 4: "Long-Haul / Large Fleet"** ($n=110,085$)
  - **Profile: Structural Difference.** Massive jump in aircraft size (~180 seats vs ~125 for others), very new planes (~7.6 years old), and long flight distances.
- **Insight:** Represents distinct wide-body/transcontinental operations.

Anomaly Detection & Monitoring

- To ensure the model remains reliable in production, a robust anomaly detection framework was implemented. By scoring the entire dataset (Train + Val + Test) with the champion GBT model, the system established baseline risk profiles. Using a **Robust Z-Score** (based on Median and MAD rather than Mean/Std Dev), the system successfully flagged specific months, time blocks, and airports that experienced statistically significant deviation in delay rates.

Aggregation: Data was aggregated into three monitoring views:

- **Monthly:** To track macro-level seasonal drift.
- **Month × TimeBlock:** To find specific "bad times of day" (e.g., evening rush hour in December).
- **Month × Airport:** To find specific hubs failing in specific months.

Detection Algorithm: Robust Z-Score.

Drift protection using robust z-scores:

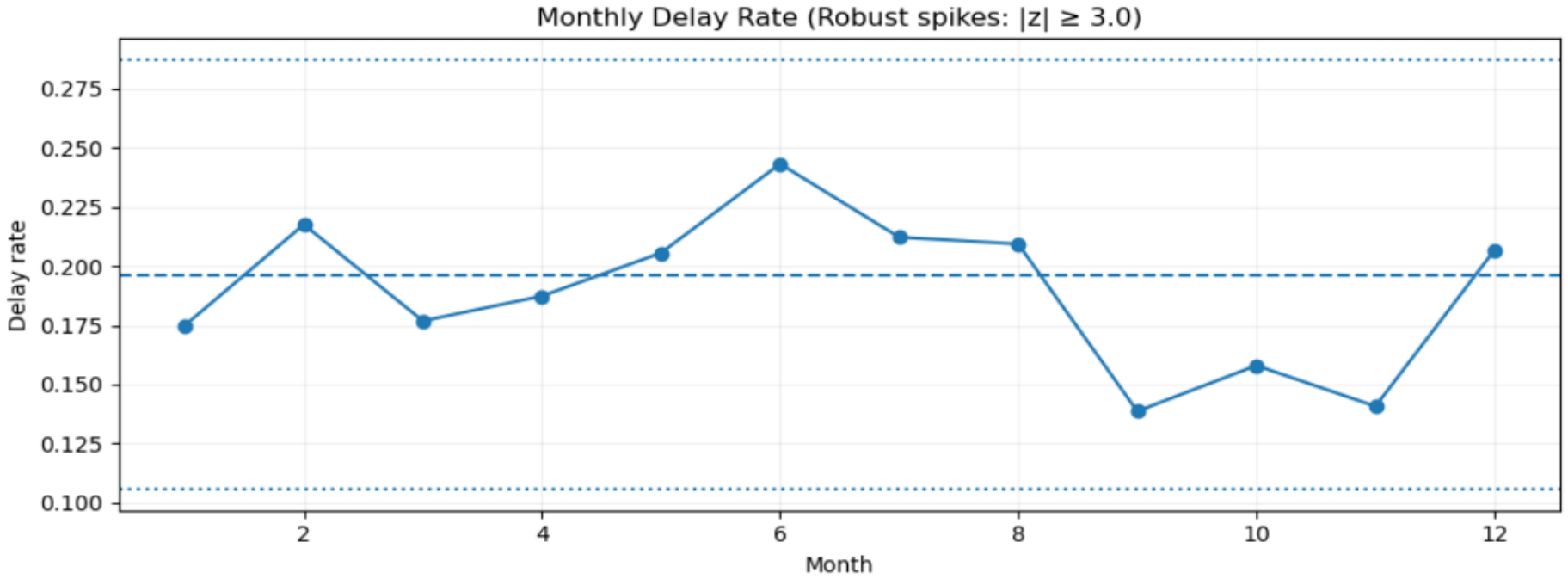
- $z = 0.6745 \cdot \frac{x - \text{median}}{\text{MAD}}$

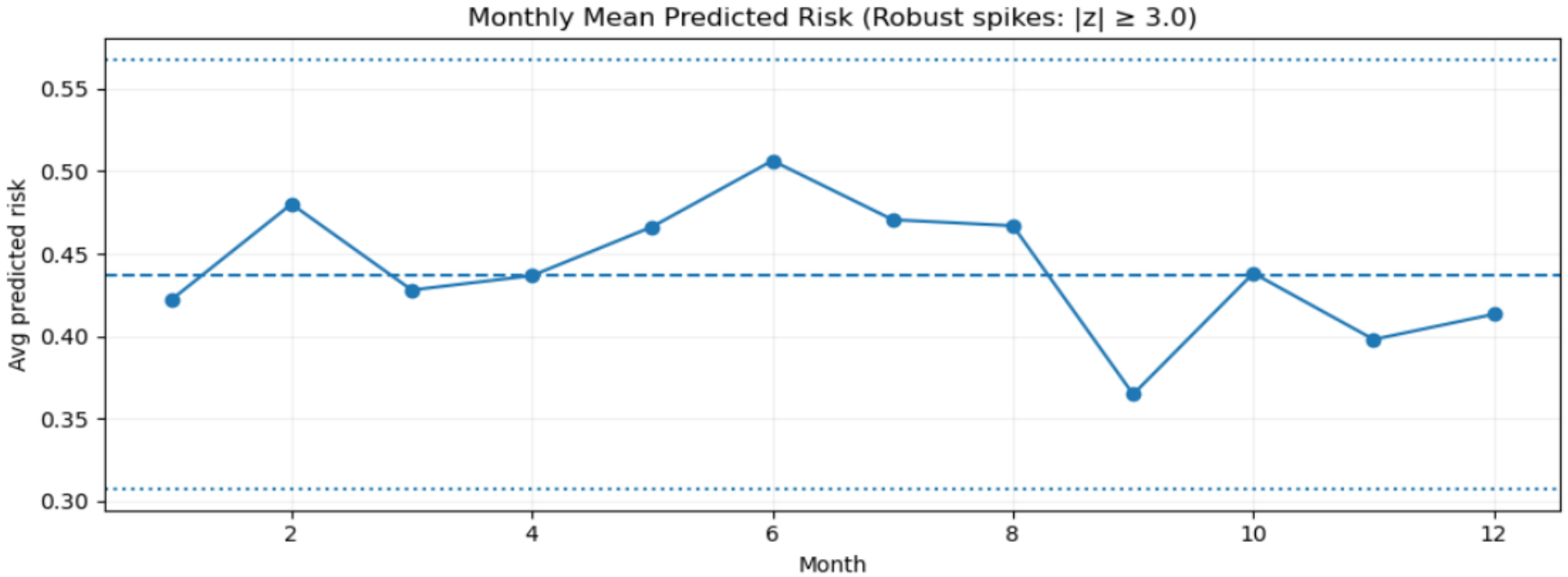
Spike scoring (robust z-score) + visuals (Matplotlib)

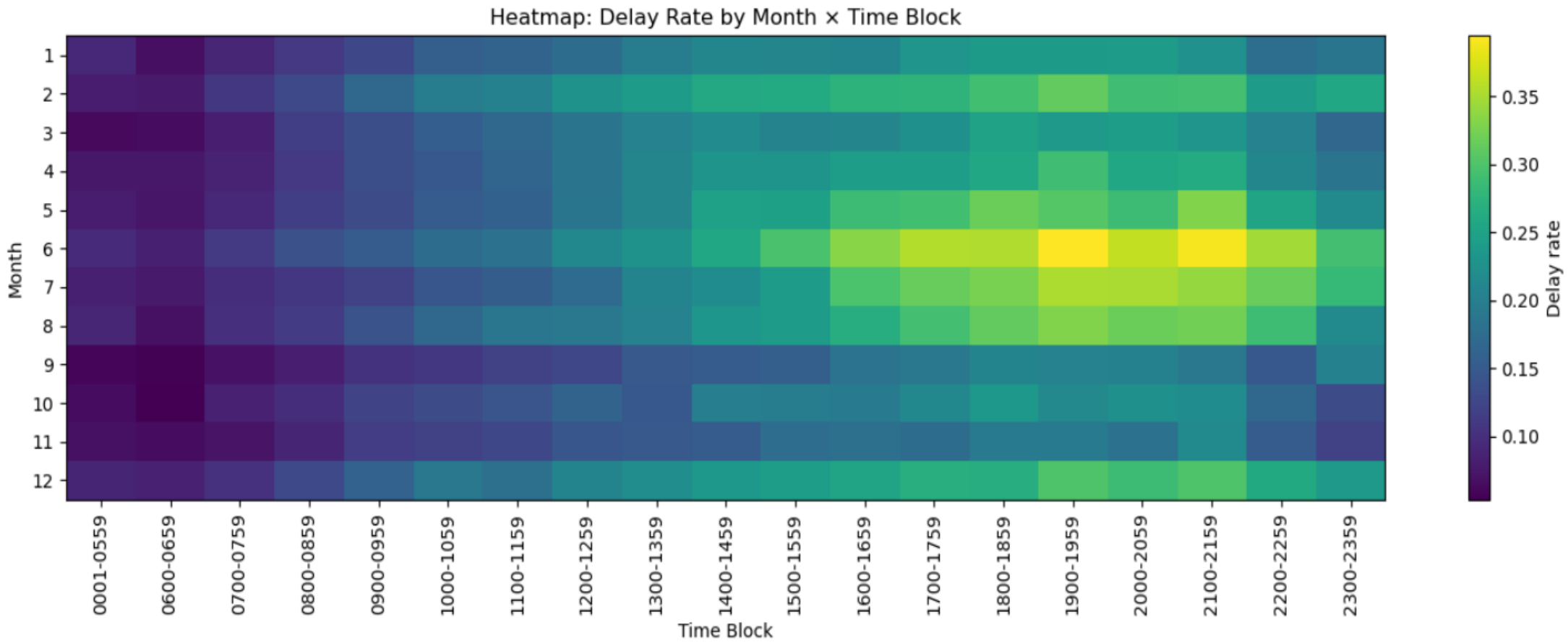
- Core anomaly logic:
- Compare each segment’s delay rate to its own baseline
- Use robust z-score (median/MAD) so outliers don’t break the detector
- Flag “spikes” as abs(z) >= 3.5

The analysis revealed distinct seasonal regimes:

- **Low Risk:** Months 1, 9, 11 (Delay rates ~13–15%).
- **High Risk:** Months 6, 7, 12 (Delay rates >20%).
- **Drift Detection:** The model's predicted risk (avg_pred_risk) tracked the actual delay rate (delay_rate) closely across months, confirming that the model adapts well to seasonality and is not "drifting" (losing accuracy) blindly.


Monthly Delay Rate (Robust spikes: |z| ≥ 3.0)
Delay rate
0.275
0.250
0.225
0.200
0.175
0.150
0.125
0.100
2
4
6
8
10
12
Month


Monthly Mean Predicted Risk (Robust spikes: |z| ≥ 3.0)
Avg predicted risk
0.55
0.50
0.45
0.40
0.35
0.30
2
4
6
8
10
12
Month



Top Month×TimeBlock spikes (within-block robust z):

	MONTH	DEP_TIME_BLK	n	delay_rate	avg_pred_risk	z_delay_blk
227	11	1400-1459	2938	0.153165	0.456286	-2.540414
117	9	1400-1459	3116	0.152118	0.401222	-2.577817
216	11	1300-1359	2850	0.148772	0.428502	-3.198366
19	9	1300-1359	2958	0.148073	0.382264	-3.237352
147	10	1300-1359	2934	0.146898	0.468050	-3.302872

Model Calibration Insight

- The model assigns relatively high predicted delay probabilities across many airports, while the observed delay rates are significantly lower, indicating systematic over-prediction.
- Major hubs such as Chicago O'Hare and JFK are flagged as high risk, with predicted delay rates nearly double the actual delay frequencies, resulting in many false positives.
- Despite this pessimistic bias, the model effectively ranks airports by relative delay risk, distinguishing higher-risk locations from lower-risk ones.
- Delay risk increases throughout the day, and the model correctly reflects this pattern by assigning higher predicted risk to late afternoon and evening departures.
- Overall, the model captures meaningful temporal and spatial delay patterns but requires calibration or threshold adjustment to reduce over-flagging in operational use.

Scaling Analysis

- Runtime initially improves as the number of workers increases, demonstrating the benefits of parallel computation in a distributed environment.
- The lowest total runtime is achieved with four workers, indicating an optimal balance between parallelism and system overhead.
- Increasing the cluster size beyond four workers results in degraded performance rather than further speedup.
- This slowdown is primarily caused by increased communication, synchronization, and data shuffling overhead across the cluster.
- These results highlight that scaling out must be carefully tuned, as additional resources do not always translate into better performance.

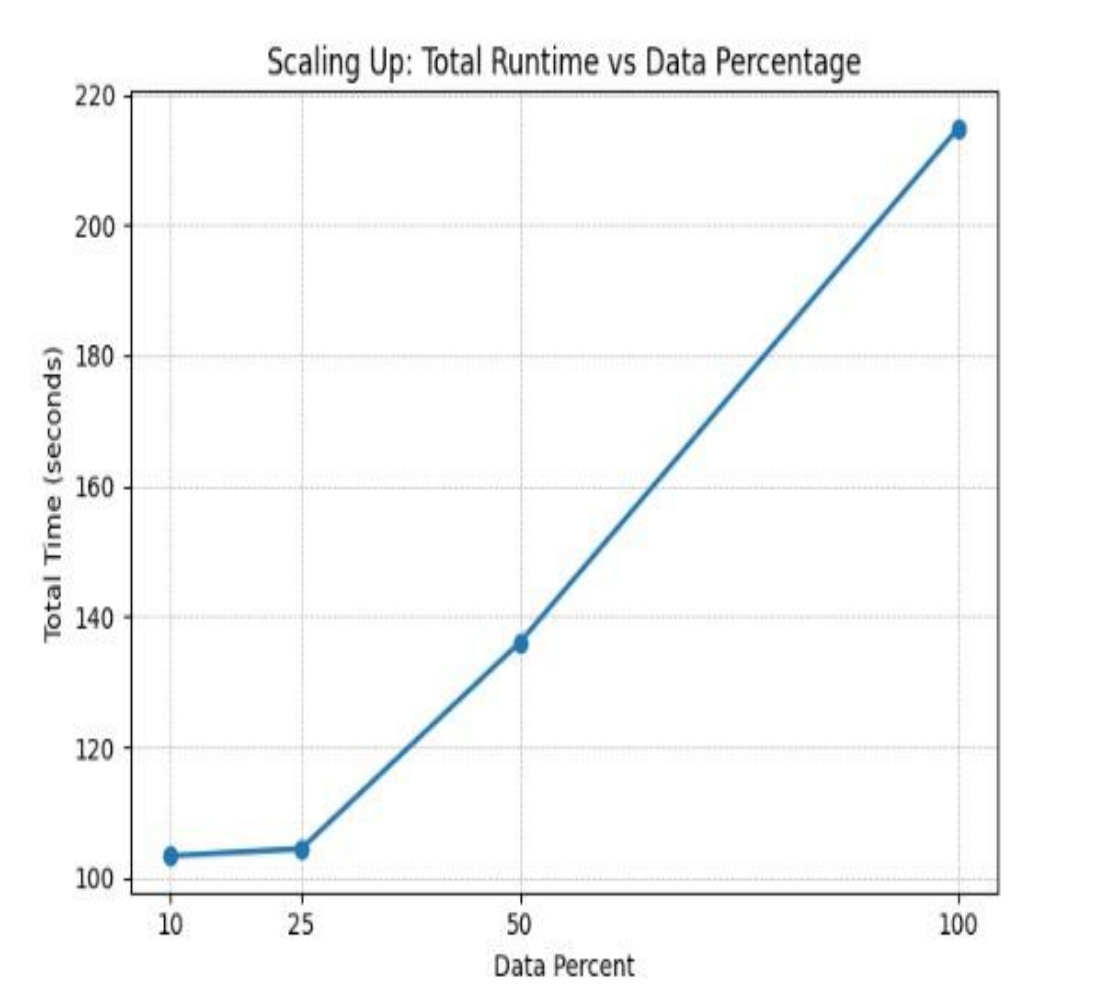
Scale up

- Scale-up: Data Volume ROI
- Trained on data fractions: 10%, 25%, 50%, 100%
- Result:

➢ PR-AUC improves ~5.5% relative from 10% to 100% (0.289 → 0.305)

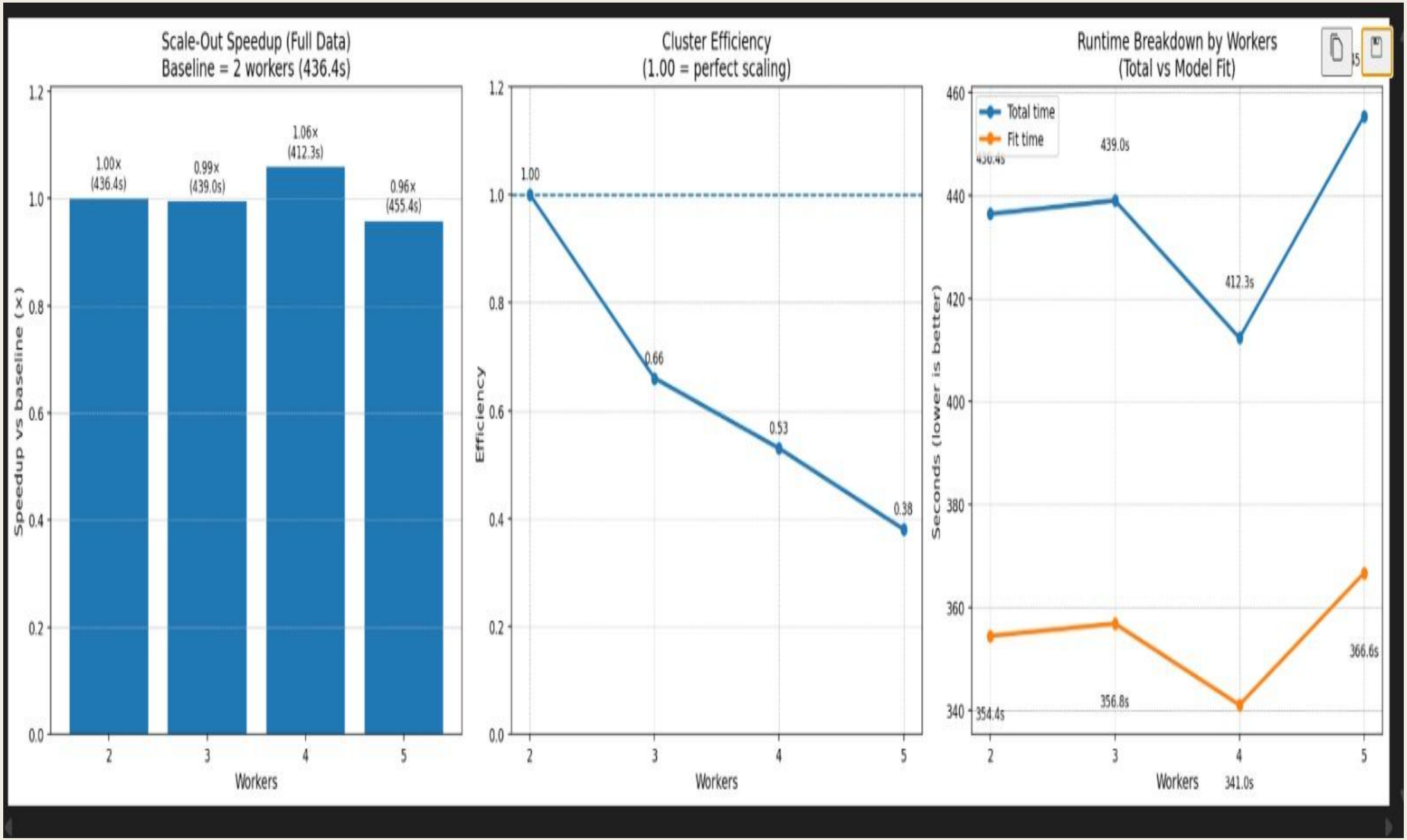
➢ Training time scales roughly linearly (~87s → ~194s)

➢ So full data helps capture rare "long-tail" patterns.



Scale out

- Scale-Out: Cluster Capacity & The "Break-Even Point"
- We tested horizontal scaling by increasing the cluster size from 2 to 5 worker nodes (w_2 to w_5) using the standard n1-standard-2 machine type.
- Observation ("Negative Scaling"): Total runtime increased slightly as more workers were added.
  - 2 Workers (Baseline): ~436 seconds total runtime.
  - 5 Workers (Max Scale): ~455 seconds total runtime.
- Metric Analysis:
  - **Speedup Factor:** The actual speedup line remained flat or slightly negative, failing to match the ideal linear growth curve.
  - **Cluster Efficiency**: The 5-worker cluster operated at <40% efficiency compared to the baseline.
  - **Technical Diagnosis (The Overhead Principle):**
  - **I/O vs. CPU Bound:** The GBT configuration (maxIter=80) was not computationally heavy enough to saturate the CPUs of 5 nodes.
  - **Network Serialization:** The time saved by parallelizing the math was outweighed by the network latency required to "shuffle" data partitions between 5 different machines.
  - **Final Verdict:** We identified the 2-Worker Cluster as the operational Break-Even Point. Scaling beyond this yields diminishing returns for this specific dataset size (<50GB). A compact cluster minimizes overhead and maximizes ROI.


Scale-Out Speedup (Full Data)
Baseline = 2 workers (436.4s)
Speedup vs baseline (×)
1.00×
(436.4s)
0.99×
(439.0s)
1.06×
(412.3s)
0.96×
(455.4s)
Workers
Cluster Efficiency
(1.00 = perfect scaling)
Efficiency
1.00
0.66
0.53
0.38
Workers
Runtime Breakdown by Workers
(Total vs Model Fit)
Total time
Fit time
Seconds (lower is better)
439.0s
412.3s
366.6s
356.8s
354.4s
341.0s
Workers

Conclusion & Future Work

- The project demonstrates effective delay prediction using Big Data techniques. Spark enables scalable and reliable modeling.
- Advanced feature engineering significantly improved model performance. Leakage prevention was critical.
- Gradient Boosted Trees achieved the best results. They handled skew and non-linearity well.
- Scaling experiments revealed practical system limits. Bigger clusters are not always better.
- Future work includes real-time prediction and deep learning models. This can further enhance operational impact.